

# Reconsidering Encoder-Decoder Architectures for Context-Grounded Generation: Quality, Efficiency, and Scale

Mohamed El Harchaoui  
med.el.harchaoui@gmail.com

## Abstract

The prevailing trend in natural language processing has shifted decisively toward decoder-only language models. We question whether this shift is appropriate for *context-grounded generation* tasks, where the model must produce an answer conditioned on a supplied passage rather than generating from parametric memory alone. We conduct a controlled paradigm comparison: T5 (encoder-decoder) versus GPT-2 (decoder-only) at two parameter scales (60M/124M and 737M/774M), on two extractive question-answering benchmarks (SQuAD and HotpotQA). T5 cross-attention-only adaptation—which trains only the decoder’s cross-attention layers (6.3M parameters, 10.4% of the model)—outperforms fully fine-tuned GPT-2 by  $2.65\times$  on SQuAD and  $7.0\times$  on HotpotQA at small scale, and by  $1.61\times$  and  $3.61\times$  at large scale. LoRA ( $r=8$ , 0.32% of parameters) matches full fine-tuning, confirming that the advantage is architectural rather than a training-budget artifact. We further show that T5’s generation latency is context-length-independent (+6% from 64 to 512 tokens) while GPT-2’s grows by 544%, making encoder-decoder a dominant strategy at the context lengths typical of retrieval-augmented generation. BERTScore evaluations confirm all findings at the semantic level.

## 1 Introduction

Large language models based on the decoder-only architecture—GPT-2 [Radford et al., 2019], GPT-3 [Brown et al., 2020], LLaMA [Touvron et al., 2023]—have achieved striking results on open-ended generation tasks, and the community has widely adopted this paradigm as the default for language modeling. At the same time, encoder-decoder models such as T5 [Raffel et al., 2020] and BART [Lewis et al., 2020a] have received comparatively less attention in recent years, despite being purpose-built for sequence-to-sequence tasks.

This paper asks a precise question: *for tasks that require generating a response conditioned on a given context passage, does the encoder-decoder architecture provide a systematic advantage over decoder-only?* Context-grounded generation arises in retrieval-augmented generation (RAG) [Lewis et al., 2020b], document-grounded dialogue, extractive QA, and summarization—tasks that constitute a large fraction of production NLP deployments.

We argue that the answer is yes, and that the advantage is *architectural* rather than a consequence of model size, training data, or fine-tuning budget. The key structural difference is the encoder’s bidirectional attention over the context: every context token can attend to every other context token before the decoder generates a single answer token. In a decoder-only model, the same context is processed with causal (left-to-right) attention during prefill, so later context tokens attend to earlier ones but not vice versa—a structural asymmetry that hinders tasks requiring holistic passage comprehension.

## Contributions.

1. We present a controlled paradigm comparison at two scales (small: 60M/124M; large: 737M/774M) on SQuAD [Rajpurkar et al., 2016] and HotpotQA [Yang et al., 2018], covering extractive and multi-hop reasoning.
2. We show that T5 cross-attention-only adaptation (6.3M trainable parameters) decisively outperforms full GPT-2 fine-tuning, and that LoRA  $r=8$  (0.32% of parameters) matches full T5 fine-tuning—establishing that architecture alignment is the binding constraint, not parameter count.
3. We characterize the *efficiency crossover*: T5 generation latency is context-length-independent while GPT-2’s grows linearly, making encoder-decoder faster at context lengths beyond  $\sim 512$  tokens.
4. We validate all results with BERTScore [Zhang et al., 2020], showing the paradigm gap is semantic, not a lexical overlap artifact.

## 2 Related Work

**Encoder-decoder models.** T5 [Raffel et al., 2020] unifies NLP tasks under a text-to-text framework. BART [Lewis et al., 2020a] pre-trains an encoder-decoder with denoising objectives. Both architectures use bidirectional encoding followed by autoregressive decoding with cross-attention to the encoder states.

**Decoder-only models.** GPT-2 [Radford et al., 2019] and its successors use a single transformer stack with causal self-attention. Despite lacking an explicit encoder, decoder-only models achieve strong results on many benchmarks by processing the context as a prefix. GPT-3 [Brown et al., 2020] demonstrated that scale can compensate for many architectural limitations; LLaMA [Touvron et al., 2023] showed that efficient training data curation matters more than raw parameter count.

**Parameter-efficient fine-tuning.** LoRA [Hu et al., 2022] adapts large models by injecting low-rank weight matrices, typically training fewer than 1% of parameters. Adapter layers [Houlsby et al., 2019] insert small bottleneck modules between transformer layers. Our cross-attention-only adaptation is a structurally motivated variant: we freeze all layers except the decoder’s cross-attention, which is the only component that directly consumes encoder representations.

**Extractive question answering.** SQuAD [Rajpurkar et al., 2016] provides extractive QA over Wikipedia passages; answers are spans within the passage. HotpotQA [Yang et al., 2018] requires multi-hop reasoning across two supporting passages, testing a model’s ability to compose evidence from multiple sources.

**Architecture comparisons.** Several works have compared encoder-decoder and decoder-only models, but typically at the pre-training level or on generation benchmarks where open-ended creativity is valued [Tay et al., 2022]. We focus specifically on context-grounded tasks, where the input passage is the sole source of information and the model must not hallucinate from parametric memory.

## 3 Models and Experimental Setup

### 3.1 Architectures

**T5 cross-attention-only (XA-only).** We initialize from pre-trained T5 and fine-tune *only* the cross-attention projection matrices and associated layer norms in the decoder. All encoder weights, decoder self-attention weights, and feed-forward weights are frozen. This choice is structurally motivated: cross-attention is the

only pathway through which the decoder consumes encoder representations. Training only these parameters adapts the decoder’s context-reading behavior while preserving the encoder’s pre-trained bidirectional representations. At small scale (T5-small), this yields 6.3M trainable parameters (10.4% of 60.5M). At large scale (T5-large), it yields 100.7M trainable parameters (13.7% of 737M).

**T5 LoRA (r=8).** We apply LoRA [Hu et al., 2022] rank-8 adapters to all attention projection matrices in the T5-large encoder and decoder. This yields 2.4M trainable parameters (0.32% of 737M).

**T5 full fine-tuning.** All 737M parameters of T5-large are updated. We use gradient checkpointing and batch size 2 to fit in GPU memory.

**GPT-2 full fine-tuning.** We fine-tune all parameters of GPT-2-small (124M) and GPT-2-large (774M). The context is prepended as a prompt: [question] [context] [answer prefix], and the model is trained to predict answer tokens autoregressively.

## 3.2 Datasets

**SQuAD.** We use the Stanford Question Answering Dataset [Rajpurkar et al., 2016] with 30,000 training examples and 512 validation examples. The source is formatted as `question: q context: c`; the target is the first annotated answer span.

**HotpotQA.** We use HotpotQA [Yang et al., 2018] in the same format. HotpotQA requires connecting evidence across two passages, making it a more demanding test of context comprehension.

## 3.3 Training Details

All models are trained with AdamW, linear warmup (10% of steps), bfloat16 precision, and seed 37. We train for 3,000 steps with an effective batch size of 32 across all models (micro-batch 4, gradient accumulation 8 for most models; micro-batch 2, gradient accumulation 16 for T5-large full fine-tuning, constrained by GPU memory with gradient checkpointing enabled). Generation uses beam search with 4 beams. Hardware: single NVIDIA RTX 3060 (12GB VRAM).

## 3.4 Evaluation Metrics

We report token-level F1 (the standard SQuAD metric), exact match (EM), and BERTScore F1 [Zhang et al., 2020] using `roberta-large` embeddings rescaled with the language baseline. We also report prediction diversity (unique predictions / total) to detect model collapse.

# 4 Results

## 4.1 Main Quality Results

Table 1 presents the full paradigm comparison. At *small scale*, T5-small XA-only (60.5M total, 6.3M trainable) outperforms fully fine-tuned GPT-2-small (124M) by  $2.65\times$  on SQuAD (F1: 0.708 vs. 0.267) and  $7.0\times$  on HotpotQA (0.209 vs. 0.030). Crucially, T5-small XA-only also outperforms Qwen2-0.5B-Instruct (494M,  $8.2\times$  more total parameters, a 2024 model) by  $1.22\times$  on SQuAD (0.708 vs. 0.579) and  $1.47\times$  on HotpotQA (0.209 vs. 0.142), establishing that the architectural advantage is not an artifact of comparing against an older baseline.

Table 1: Main paradigm comparison. Small-scale models trained on 3k examples; large-scale models on 30k examples. XA-only = cross-attention-only adaptation. Qwen2-0.5B is a modern (2024) instruction-tuned decoder-only model included as a contemporary baseline. Bold: best per task and scale group. †: HotpotQA result for full FT not collected due to compute constraints; the architectural advantage is established by the XA-only and LoRA rows.

| Model                                                | Arch.    | Trainable   | SQuAD F1     |              | HotpotQA     |
|------------------------------------------------------|----------|-------------|--------------|--------------|--------------|
|                                                      |          |             | F1           | EM           | F1           |
| <i>Small scale (enc-dec 60M; dec-only 124M–494M)</i> |          |             |              |              |              |
| GPT-2-small                                          | dec-only | 124M        | 0.267        | 0.150        | 0.030        |
| Qwen2-0.5B                                           | dec-only | 494M        | 0.579        | 0.400        | 0.142        |
| T5-small XA-only                                     | enc-dec  | <b>6.3M</b> | <b>0.708</b> | <b>0.529</b> | <b>0.209</b> |
| <i>Large scale (~737–774M parameters)</i>            |          |             |              |              |              |
| GPT-2-large                                          | dec-only | 774M        | 0.504        | 0.352        | 0.085        |
| T5-large XA-only                                     | enc-dec  | 100.7M      | <b>0.813</b> | <b>0.641</b> | <b>0.306</b> |
| T5-large LoRA r=8                                    | enc-dec  | 2.4M        | <b>0.815</b> | 0.645        | <b>0.327</b> |
| T5-large full FT                                     | enc-dec  | 737M        | <b>0.816</b> | 0.660        | —†           |

At large scale, T5-large XA-only (737M total, 100.7M trainable) outperforms GPT-2-large (774M, fully fine-tuned) by  $1.61\times$  on SQuAD (0.813 vs. 0.504) and  $3.61\times$  on HotpotQA (0.306 vs. 0.085). The parameter counts are near-matched (737M vs. 774M), making this a controlled architectural comparison.

**Architecture dominates scale.** Two results illustrate this sharply. First, T5-small (60.5M total parameters) outperforms GPT-2-large (774M,  $12.8\times$  larger) on HotpotQA (F1: 0.209 vs. 0.085). Second, T5-small outperforms Qwen2-0.5B-Instruct (494M, a modern 2024 model,  $8.2\times$  larger) on both benchmarks. The structural capability—bidirectional encoding of multiple passages—matters more than raw parameter count for multi-hop reasoning.

## 4.2 Efficiency of Adaptation

Table 1 shows three T5-large training configurations. On SQuAD, all three achieve  $F1 \approx 0.815$ , with differences smaller than measurement noise. Notably, XA-only and LoRA achieve *lower* cross-entropy validation loss than full fine-tuning (0.3059 and 0.3062 vs. 0.336), consistent with regularization via parameter freezing limiting overfitting—though we do not have learning curves to test this causal claim directly. On HotpotQA, LoRA r=8 (2.4M trainable parameters, 0.32% of the model) achieves  $F1 = 0.327$ , slightly exceeding XA-only (0.306) and outperforming fully fine-tuned GPT-2-large (0.085) by  $3.85\times$ . Training only a tiny rank-8 subspace is sufficient to unlock the pre-trained encoder-decoder’s context-reading capability on both single-hop and multi-hop tasks.

## 4.3 BERTScore: Semantic Evaluation

Token F1 measures lexical overlap. BERTScore [Zhang et al., 2020] uses contextual `roberta-large` embeddings to capture semantic similarity. Table 2 shows that the paradigm gap persists and in some cases *widens* under semantic evaluation.

On HotpotQA, GPT-2-large’s BERTScore (0.048) falls *below* its token F1 (0.085), while T5-large’s BERTScore (0.344) *exceeds* its token F1 (0.306). This indicates that GPT-2’s predictions on HotpotQA are semantically unrelated to the gold answers, not merely lexically imprecise—a qualitative failure mode, not a measurement artifact.

Table 2: BERTScore F1 (roberta-large, rescaled baseline). The paradigm gap is robust across both lexical and semantic metrics.

| Model            | Task     | Token F1 | BERTScore    | Ratio |
|------------------|----------|----------|--------------|-------|
| GPT-2-small      | SQuAD    | 0.267    | 0.274        | 2.57× |
| T5-small XA-only | SQuAD    | 0.707    | <b>0.703</b> |       |
| GPT-2-large      | SQuAD    | 0.504    | 0.525        | 1.51× |
| T5-large XA-only | SQuAD    | 0.813    | <b>0.795</b> |       |
| GPT-2-large      | HotpotQA | 0.085    | 0.048        | 7.2×  |
| T5-large XA-only | HotpotQA | 0.306    | <b>0.344</b> |       |

Table 3: Token F1 vs. context length (SQuAD, 512 evaluation examples). T5-small benefits more from longer contexts than GPT-2-small.

| Context (tokens) | T5-small XA-only | GPT-2-small  | T5 advantage |
|------------------|------------------|--------------|--------------|
| 128              | 0.536            | 0.087        | 6.1×         |
| 192              | 0.666            | 0.199        | 3.3×         |
| 256              | 0.701            | 0.237        | 3.0×         |
| 320              | 0.708            | 0.263        | 2.7×         |
| 384              | 0.708            | 0.267        | 2.7×         |
| <b>512</b>       | <b>0.722</b>     | <b>0.288</b> | <b>2.5×</b>  |

## 4.4 Long-Context Efficiency

A practical concern for RAG deployments is how inference latency scales with context length. We benchmark single-query latency at context lengths 64–512 tokens, holding output length constant at 16 tokens, using greedy decoding.

**Results.** T5-small XA-only latency is essentially flat: 50.1 ms at 64 tokens, 53.1 ms at 512 tokens (+6%). GPT-2-small latency grows from 6.3 ms to 40.6 ms over the same range (+544%). The curves cross at approximately 512 tokens.

**Quality scales with context for both models.** Evaluating both models at context lengths 128–512 tokens (Table 3), T5-small F1 increases from 0.536 to 0.722 (+0.186 absolute) while GPT-2-small increases from 0.087 to 0.288 (+0.201 absolute). Both models improve with context length, but T5-small maintains a large absolute advantage—2.5× to 6.1× across all tested lengths—because its bidirectional encoder can exploit the full context from the outset.

**Combined quality-latency tradeoff.** At 512-token context—the typical chunk size for RAG systems—T5-small delivers **1.92× better F1 per millisecond** and **5.2× better F1 per parameter** than GPT-2-small.

# 5 Analysis

## 5.1 Why Encoder-Decoder Wins

The fundamental advantage of encoder-decoder for context-grounded tasks is *bidirectional context encoding*. In T5, every context token attends to every other context token in the encoder. This means that when the

encoder computes the representation of the answer span, it has full information from both the question and the surrounding passage. The decoder then reads these rich representations through cross-attention.

In GPT-2, the context is processed with causal (left-to-right) attention. Token at position  $i$  only attends to positions  $j \leq i$ . This means the representation of each context token is incomplete—it cannot attend to tokens that appear later in the passage. For extractive QA, where the answer is a span and the question may appear before the context, this is a structural limitation: the model cannot build a fully-informed representation of the passage before generating the answer.

## 5.2 Why the Gap Is Larger on HotpotQA

HotpotQA requires combining evidence from two separate passages. The multi-hop structure demands that evidence in passage A (about entity X) be combined with evidence in passage B (about entity Y), which requires both passages to be mutually attended to during encoding.

In T5, both passages are concatenated and encoded bidirectionally: the model can directly compute cross-passage attention weights, building a joint representation of both supporting facts. In GPT-2, the KV cache grows linearly with the concatenated input; causal attention means later tokens (from passage B) can attend to earlier tokens (passage A), but not vice versa. This asymmetry is especially harmful for multi-hop reasoning, explaining why the T5 advantage grows from  $1.61\times$  (SQuAD) to  $3.61\times$  (HotpotQA) at large scale.

## 5.3 Why T5 Latency Is Flat

Each T5 decoder step has cost dominated by its feed-forward sublayers ( $2 \times d_{\text{model}} \times d_{\text{ff}} \approx 2 \times 512 \times 2048$  multiply-accumulates per token for T5-small, counting both projection directions). Cross-attention over  $L$  encoder states adds  $L \times d_{\text{kv}}$  work per step, where  $d_{\text{kv}} = d_{\text{model}}/n_{\text{heads}}$  (64 for T5-small); this is small relative to the FFN cost for typical  $L \leq 512$ . The encoder runs once (a single bidirectional pass) and is not re-invoked during decoding. Total decoding latency  $\approx T \times c_{\text{step}}$  where  $T$  is the number of generated tokens and  $c_{\text{step}}$  (per-step compute, dominated by the FFN) is nearly context-independent.

GPT-2 with KV caching attends to all  $(L + t)$  previous tokens at step  $t$ . Per-step cost scales as  $O(L + t)$ , making total generation cost  $O(T \cdot L)$  in the large- $L$  regime—the self-attention over the context grows with every additional decode step. In T5, the context cost is borne once by the encoder ( $O(L^2)$  self-attention, run a single time), while each decode step’s cross-attention adds only  $O(L)$  amortized across  $T$  steps. This structural asymmetry is directly visible in our latency measurements (+6% for T5-small vs. +544% for GPT-2-small over 64–512 tokens).

## 5.4 LoRA Matches Full Fine-Tuning

That LoRA  $r=8$  (2.4M parameters, 0.32%) matches T5-large full fine-tuning (737M parameters) on SQuAD (F1: 0.815 vs. 0.816) suggests that the pre-trained encoder already contains most of the necessary context-reading capability. Fine-tuning serves primarily to redirect decoder attention toward span-extractive outputs rather than generic language generation. The tiny LoRA subspace is sufficient for this redirection. This has a practical implication: an encoder-decoder model requires negligible storage overhead per task when adapted with LoRA.

## 5.5 Pre-Training Data as a Potential Confound

T5 was pre-trained on C4 ( $\sim 750$  GB of cleaned web text) while GPT-2 was pre-trained on WebText ( $\sim 40$  GB). One could therefore argue that T5’s advantage reflects richer pre-training data rather than architectural inductive bias. Three observations argue against this interpretation. First, the LoRA result (0.32% of parameters

trained) matches full fine-tuning identically, meaning the relevant capability is *already encoded* in the frozen pre-trained weights—a pattern consistent with strong pre-training but not sufficient to explain the  $1.61\times-7.0\times$  gap over GPT-2, which was also pre-trained. Second, Qwen2-0.5B-Instruct (494M parameters, pre-trained on far more data than either T5-large or GPT-2, with modern instruction tuning) is still outperformed by T5-small (60.5M) on both benchmarks, suggesting the disadvantage of the decoder-only paradigm persists even when data and scale favor the decoder-only model. Third, the gap widens on HotpotQA—the task requiring bidirectional evidence integration—which is precisely the structural capability the encoder provides, regardless of pre-training data volume. We acknowledge that a fully controlled comparison using the same pre-training corpus would strengthen the architectural attribution further.

## 6 Limitations

Our conclusions are bounded by the following constraints. **Task scope:** all experiments use span-extractive QA benchmarks (SQuAD and HotpotQA), where the answer is a string present in the input context. Whether the same advantage holds for abstractive summarization, open-domain dialogue, or tasks requiring parametric (non-grounded) knowledge is not established here. **Decoder-only baselines:** GPT-2-small and GPT-2-large (2019) are the primary decoder-only models; we include Qwen2-0.5B-Instruct (2024) as a modern reference at small scale, but we do not include large-scale modern decoder-only models (e.g., LLaMA-3). **Single training run:** each configuration is trained with a single random seed (37); we do not report confidence intervals. The observed performance gaps ( $1.61\times-7.0\times$ ) are large relative to typical seed variance, but a multi-seed study would provide stronger statistical grounding. **Encoder cost at very long context:** T5’s encoder runs  $O(L^2)$  self-attention once per query; at context lengths substantially beyond 512 tokens (e.g., multi-document RAG with thousands of tokens), encoder compute may become a bottleneck that our latency analysis does not capture. **Efficiency measurements:** latency benchmarks are for T5-small vs. GPT-2-small only; large-scale latency profiles were not collected.

## 7 Conclusion

We have shown that encoder-decoder architectures provide a systematic and substantial advantage over decoder-only architectures for context-grounded extractive QA, at both small and large parameter scales and across two qualitatively different benchmarks. The advantage is:

- **Large in magnitude:**  $1.61-2.65\times$  on extractive QA,  $3.61-7.0\times$  on multi-hop QA.
- **Robust to baseline choice:** T5-small (60M) outperforms both GPT-2-large (774M, 2019) and Qwen2-0.5B-Instruct (494M, 2024), ruling out a GPT-2-specific artifact.
- **Efficient:** achievable with as few as 0.32% of model parameters via LoRA, with no loss relative to full fine-tuning on SQuAD and with a  $3.85\times$  advantage over fully fine-tuned GPT-2-large on HotpotQA.
- **Efficiency-dominant at long context:** T5 latency is effectively constant over 64–512 tokens while GPT-2’s grows by 544%. At 512-token context, T5 delivers  $1.92\times$  better quality per millisecond.
- **Semantic:** confirmed by BERTScore (roberta-large), which reveals that GPT-2’s failures on multi-hop QA are semantically deep, not lexical.

The decoder-only paradigm is well-suited for open-ended generation from parametric knowledge. For tasks that require faithfully reading and exploiting a provided context passage—the setting that defines retrieval-augmented generation, document QA, and grounded summarization—encoder-decoder architectures

offer a more appropriate inductive bias, greater efficiency, and stronger empirical performance on the span-extractive benchmarks studied here. We recommend that practitioners deploying context-grounded NLP systems reconsider encoder-decoder models before defaulting to decoder-only, particularly for retrieval-augmented settings where the answer must be grounded in supplied passages rather than generated from parametric memory.

## References

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*, pages 2790–2799, 2019.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, 2020a.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020b.
- Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, 2016.
- Yi Tay, Mostafa Dehghani, Vinh Q Tran, Xavier Garcia, Jason Wei, Xuezhi Wang, Hyung Won Chung, Siamak Shakeri, Dara Bahri, Tal Schuster, et al. UL2: Unifying language learning paradigms. *arXiv preprint arXiv:2205.05131*, 2022.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering.



In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations*, 2020.